AGH University of Krakow

Mobile App Development

Laboratory 5 Report

Static Layouts in Android

Student:	Carlos Guzman
Project name:	CarlosGuzmanLayoutsAndActions
Language:	Kotlin
Development environment:	Android Studio
Date:	April 15, 2026

1. Objective

The objective of this laboratory practice is to create static user interfaces in Android Studio using XML layout files. During the practice, different types of layouts were explored, including `RelativeLayout` and `LinearLayout`, as well as the use of interface resources such as colors, strings, and drawable images.

2. General Description

In this laboratory, an Android application project was created in Kotlin. The practice focused on editing the XML files directly in order to understand how static layouts are defined in Android. Different interface elements such as text fields, images, and buttons were positioned inside the layouts. In addition, custom colors and string resources were declared in the corresponding files under the `res` directory.

At the current stage, the project includes the following layout files:

- `activity_main.xml`
- `layout_weekend.xml`
- `layout_calculator.xml`

3. Main Layout: `activity_main.xml`

The first part of the practice consists of creating a static screen using a `RelativeLayout`. A custom background color was added and two text elements were positioned on the screen. These elements were configured using XML attributes such as width, height, text size, text style, alignment, margins, and background colors.

This layout was used to understand how elements can be arranged relative to the parent container and to each other.

3.1. Current XML Code

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/main"
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
    android:background="@color/layout_background">

    <EditText
        android:id="@+id/hello_there_edittext"
        android:layout_width="140dp"
        android:layout_height="wrap_content"
        android:text="@string/hello_string"
        android:textSize="18sp"
        android:textStyle="bold"
        android:background="@android:color/holo_green_light"
        android:textColor="@android:color/black"
        android:gravity="center"
        android:padding="6dp"
        android:layout_centerHorizontal="true"
        android:layout_centerVertical="true"
        android:layout_marginEnd="4dp" />

    <EditText
        android:id="@+id/awesome_android_edittext"
        android:layout_width="140dp"
        android:layout_height="wrap_content"
        android:text="@string/android_is_so_awesome"
        android:textSize="18sp"
        android:textStyle="bold"
        android:background="@android:color/holo_orange_dark"
        android:textColor="@android:color/black"
        android:gravity="center"
        android:padding="6dp"
        android:layout_toEndOf="@id/hello_there_edittext"
        android:layout_alignTop="@id/hello_there_edittext" />

</RelativeLayout>
```

3.2. Screenshot Placeholder

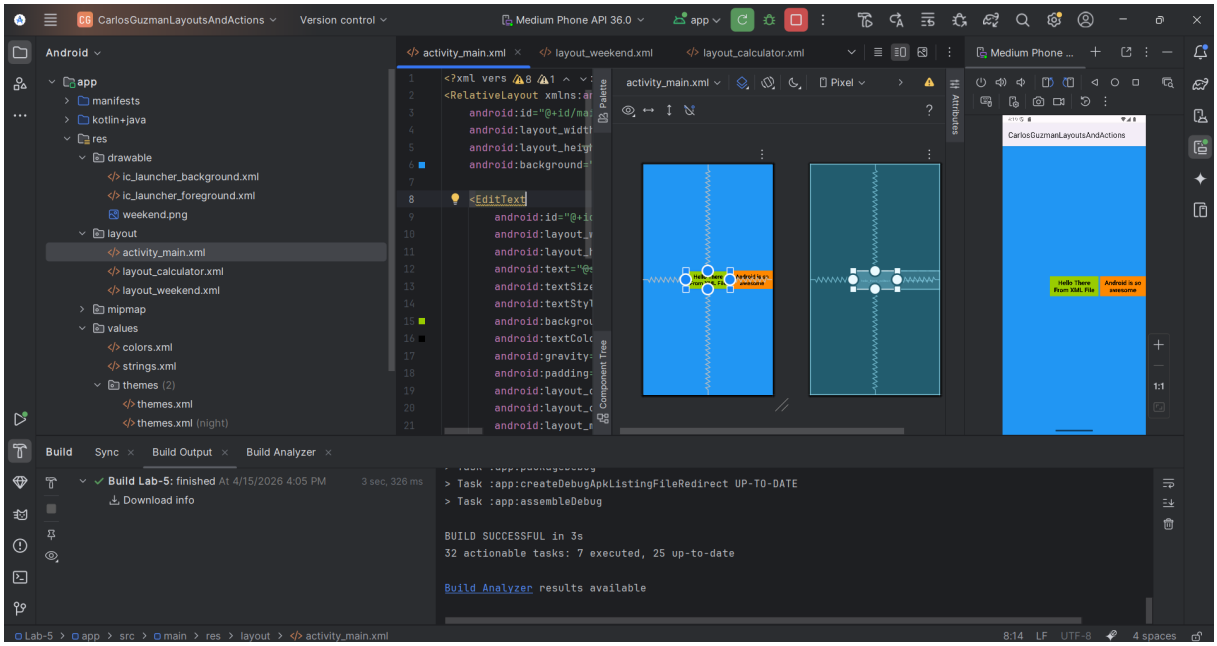

Figure 1: Main layout shown in the emulator.

4. Weekend Layout: `layout_weekend.xml`

The second part of the practice consists of creating a new XML layout file using a `LinearLayout`. In this screen, a title text and an image resource were added. This part of the laboratory helped to understand how to import drawable resources and how to organize interface elements vertically.

The image was added to the `drawable` folder and referenced inside the XML file. This layout is intended to show a simple reminder-style screen with a title and an image underneath.

4.1. Description

The layout contains:

- A vertical `LinearLayout`
- A text element at the top
- An `ImageView` using a drawable resource

4.2. Screenshot Placeholder

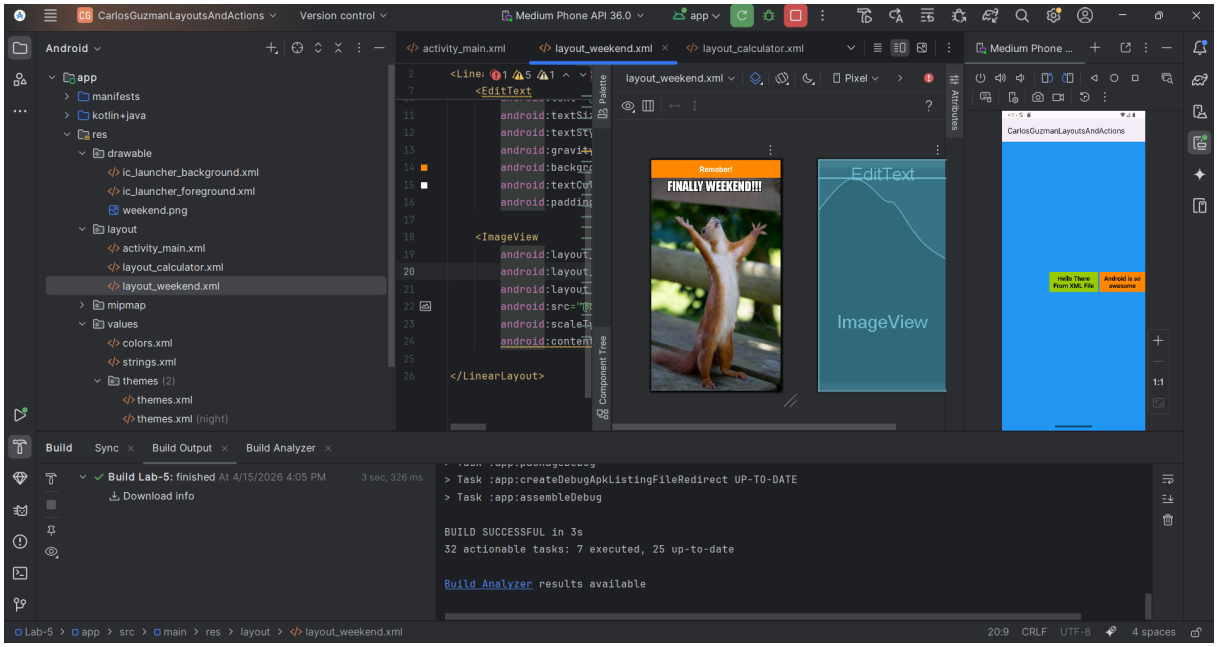

Figure 2: weekend.

5. Calculator Layout: `layout_calculator.xml`

The last part developed so far is a calculator-style interface made with XML. This layout was designed using nested `LinearLayout` containers in order to simulate rows of buttons and a display area at the top. It includes number buttons and operator buttons arranged in a grid-like structure.

This section demonstrates the use of multiple nested layouts, spacing, alignment, button styling, and a display area for the calculator.

5.1. Description

The calculator layout includes:

- A display field at the top
- Several rows of buttons
- Numeric and operator controls
- Simple visual styling for the interface

5.2. Screenshot Placeholder

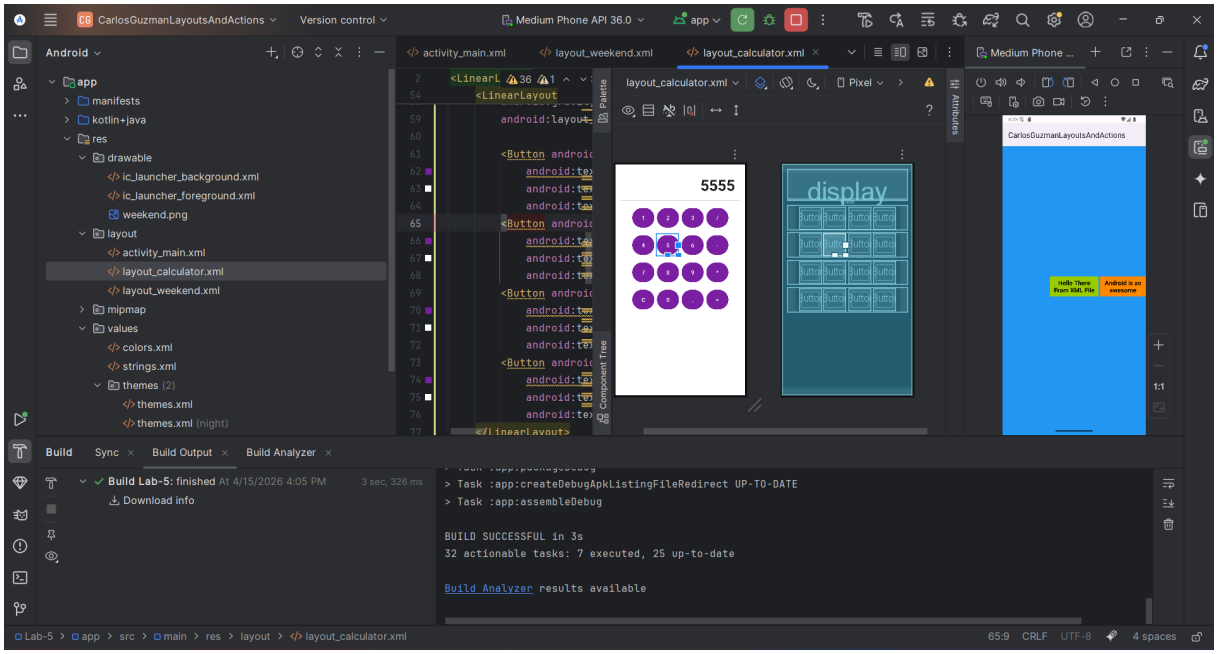

Figure 3: calculator.

6. Resources Used

The practice also required editing resource files in the `values` directory. These files help separate the interface content from the layout structure.

6.1. Colors

Custom colors were defined in `colors.xml`, for example the background color of the main layout.

6.2. Strings

Text values such as the application name and interface labels were declared in `strings.xml`. This makes the project easier to maintain and update.

6.3. Drawable

An image resource was added to the `drawable` folder and referenced from the XML file in the weekend layout.

7. MainActivity Configuration

To test each screen in the emulator, the `setContentView()` method in `MainActivity.kt` can be changed to load a specific XML layout.

For example:

```
setContentView(R.layout.activity_main)
```

```
setContentView(R.layout.layout_weekend)
```

```
setContentView(R.layout.layout_calculator)
```

This makes it possible to test each layout independently and take screenshots for the report.

8. Conclusion

In this laboratory practice, static Android layouts were created using XML files. Different types of containers were used, including `RelativeLayout` and `LinearLayout`. Resources such as colors, strings, and images were also integrated into the project.

Although the project is still being completed, the current progress already demonstrates the main concepts of XML-based interface design in Android Studio. The practice has helped reinforce the use of layout attributes, visual organization, and the relationship between resources and interface files.